Database Design and ER Model

- An Entity Relationship Diagram is a diagram that represents relationships among entities in a database.
- It is commonly known as an ER Diagram.
- An ER Diagram in DBMS plays a crucial role in designing the database.
- An Entity Relationship Diagram (ER Diagram) pictorially explains the relationship between entities to be stored in a database.
- Fundamentally, the ER Diagram is a structural design of the database.
- It acts as a framework created with specialized symbols for the purpose of defining the relationship between the database entities.
- ER diagram is created based on three principal components: entities, attributes, and relationships.

Database Design and ER Model

- Student is an entity, Teacher is an entity, similarly, Class, Subject etc are also entities.

- If a Student is an Entity, then the complete dataset of all the students will be the Entity Set

- **ER Model: Attributes & its Types**

- An attribute can be of many types, here are different types of attributes defined in ER database model:

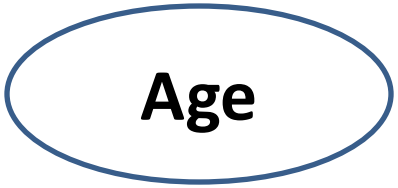
✓ **Simple attribute & composite attribute**
✓ **Single valued and Multivalued attribute**
✓ **Derived attribute**
✓ **Null attribute**

Database Design and ER Model

- **Simple attribute:**

- The attributes with **values that are atomic** and cannot be broken down further are simple attributes.

- For example, student's **age.**



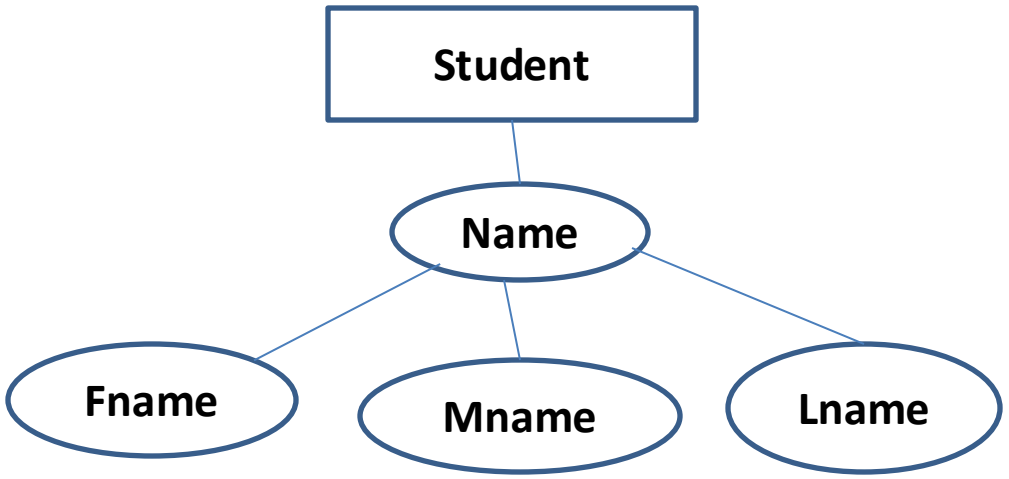
2. Composite attribute:

- A composite attribute is made up of more than one simple attribute.

The attribute can be subdivided into subparts are called as composite attribute.

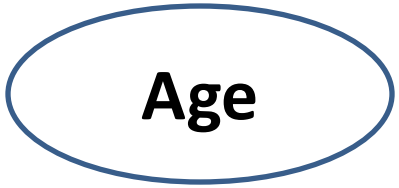
Example: Name and Address

address will contain, **house no.**, **street name**, **pincode** etc.



3. Single-valued attribute: As the name suggests, they have a single value.

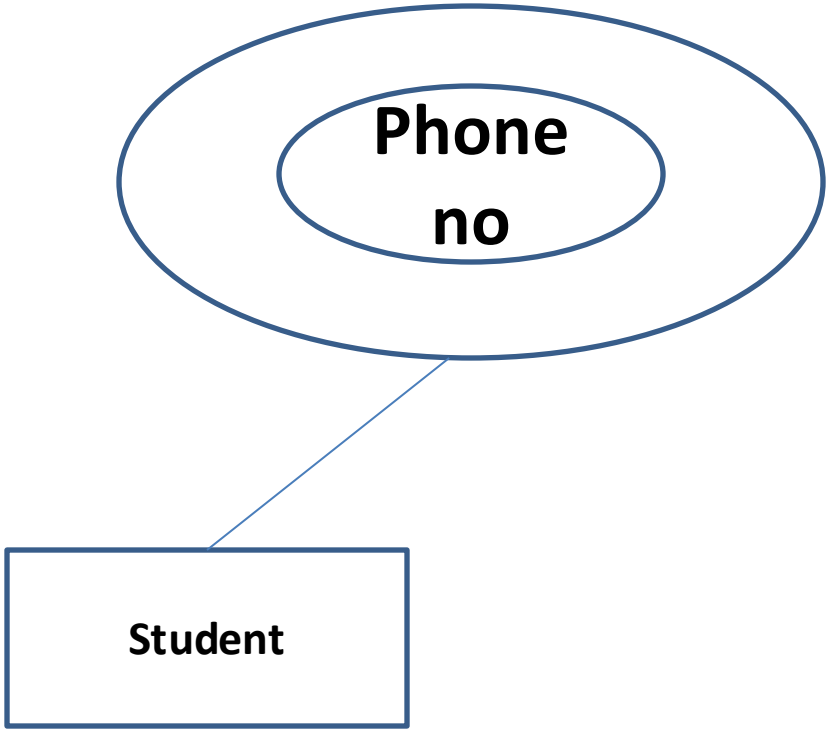
- Attributes hold a single value for single entity .
- It can be a simple or composite attribute.
- Ex: Age, DOB etc



4. Multi-valued attribute: they can have multiple values.

- Shown by double line connected to the entity in the ER diagram.
- Ex: Phone no, degrees

Database Design and ER Model



Database Design and ER Model

- **Derived Attribute:**
- These are the attributes which are not present in the whole database management system, but are derived using other attributes.

- The value for this type of attribute can be derived from the value of other related attributes or entities.

- For example, ***average age of students*** *in a class*.

- **We can calculate age from DOB & current date.**



Database Design and ER Model

Weak entity & strong entity:

Weak entity:

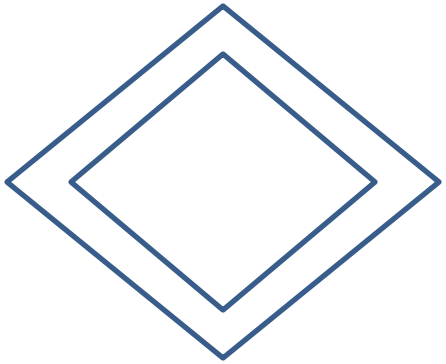
- Weak entity does not have a sufficient attribute to form or define primary key are called weak entity.
- Weak entity set depends on strong entity set.
- **Identifying relationship:** Relationship that is associating weak entity set with strong entity set.
- Weak entity set does not have any primary key.
- Denoted by double rectangle.



Database Design and ER Model

Weak entity set & strong entity set :

Strong entity set:



- Identifying relationship is denoted by double diamond.
- entity sets that have sufficient attribute to define the primary key are called strong entity set.
- Strong entity also called identifying or owner entity set.
- Strong enetity have a primary key.
- Denoted by rectangle:



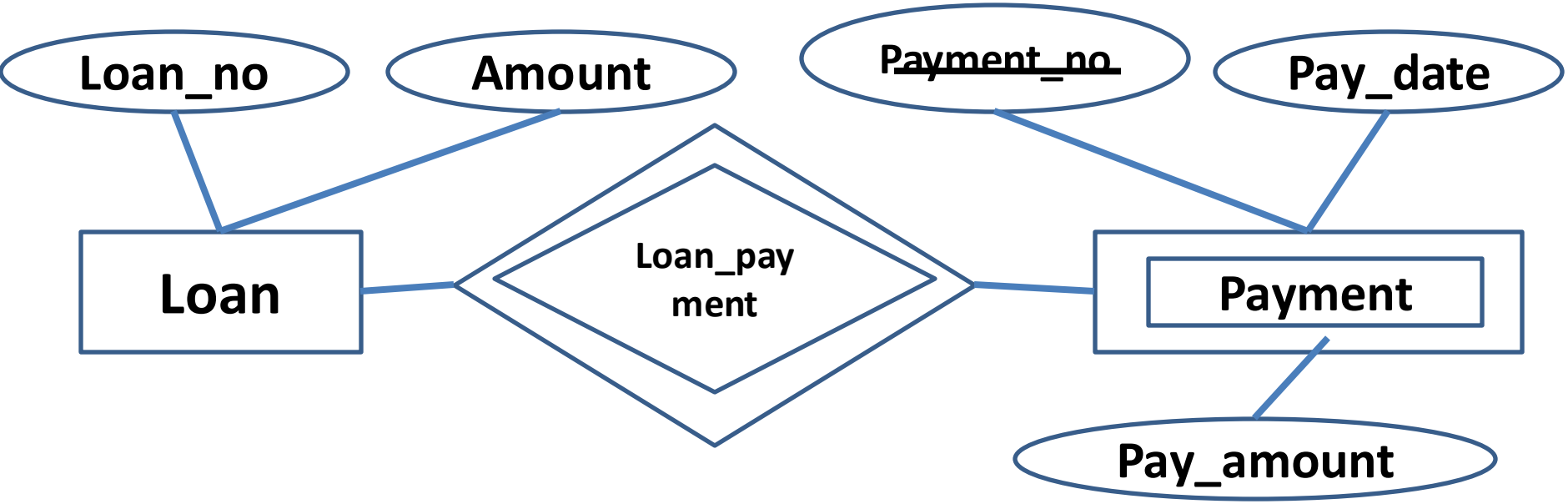


Fig: E-R diagram with weak entity set

Database Design and ER Model: Constraints

Constraints

- **Constraints :** applying certain conditions or restrictions on the database.
- Means that before inserting data into the database, we are checking for some conditions.
- If the condition we have applied to the database holds true for the data which is to be inserted, then only the data will be inserted into the database tables.
- Constraints in SQL can be categorized into two types:
- **Column Level Constraint:**
  is used to apply a constraint on a *single column*.
- **Table Level Constraint:**
  is used to apply a constraint on *multiple columns*.

Database Design and ER Model: Constraints

- **Constraints available in SQL are:**

- NOT NULL
- UNIQUE
- PRIMARY KEY
- FOREIGN KEY
- CHECK
- DEFAULT
- CREATE INDEX

- **Note : (for UNIQUE , PRIMARY KEY and FOREIGN KEY refer the following key Types point)**

Database Design and ER Model: Constraints

- **Constraints available in SQL are:**

✓ NOT NULL:

- when not null constraint is assigned on a column or a set of column, then it will not allow null values.

Create table <table Name>

(Column1 datatype(size),

Column2 datatype(size) **not null**);

- **Example:**

- create table **Emp** (EmpId number (4) not null, EmpName varchar (20) **not null**);

Database Design and ER Model: Constraints

- **Constraints available in SQL are:**

✓ **CHECK:**

The constraint defines a condition that each row must satisfy.

Single column can have multiple check conditions.

Syntax:

Column_name datatype(size) CHECK (logical Expression);

Example:

Empsalary number(20) ***CHECK***(sal>4000);

Database Design and ER Model: Constraints

- **Constraints available in SQL are:**

✓ **UNIQUE :**

- It avoids duplication of values within the rows of columns or set of columns in a table.
- It can allow null values.
- Defines on one or more columns.
- **Example: oracle**

```
CREATE TABLE Persons (
    ID int NOT NULL UNIQUE,
    LastName varchar(255) NOT NULL,
    FirstName varchar(255),
    Age int
);
```

Database Design and ER Model: Constraints

- **Constraints available in SQL are:**

✓ **UNIQUE :**

- **Example: Mysql**

Create table **student**

(

Rollno number(20),

Name varchar(30),

Unique (Rollno)

);

Database Design and ER Model: Constraints

➢ Unique Key on Multiple columns:

```
CREATE TABLE Student (
        Roll_no number(20) NOT NULL ,
        FirstName varchar(30),
        LastName varchar(255) NOT NULL,
         CONSTRAINT UC_Student UNIQUE (Roll_no, LastName)
    );
```

Database Design and ER Model: Constraints

➢ **Default:**

- Sets a default value for a column if no value is specified
- The DEFAULT constraint is used to set a default value for a column.
- The default value will be added to all new records, if no other value is specified.

- The following SQL sets a DEFAULT value for the "**Class**" column when the "**Student**" table is created:

- CREATE TABLE **Student**(
  RollNo int NOT NULL,
  LastName varchar(255) NOT NULL,
  FirstName varchar(255),

  Class varchar(255) ***DEFAULT*** 'TE A'
  );

Database Design and ER Model

Keys:

- Keys play an important role in the relational database.
- It is used to uniquely identify any record or row of data from the table.
- It is also used to *establish* and *identify* relationships between tables.
- By key , we can identify the record uniquely.

- Value of key attribute can be used to uniquely identify each tuple/row in a relation/table is called key.

- **Types of key:**
  - Primary key
  - Candidate key
  - Super key
  - Foreign key
  - Unique key
  - Composite Kay

Database Design and ER Model

- **Types of key:**
- **Primary key:**
- Primary Key is a set of attributes/columns (or attribute/columns) which uniquely identify the tuples/rows in relation or table.
- It does not allow ***null value*** and ***duplicate value***.
- The primary key is a minimal super key, so ***there is one and only one primary key in any relationship***.
- **For example,:**
- Student{ID, Aadhar_ID, F_name, M_name, L_name, Age}

Database Design and ER Model: Constraints

PRIMARY KEY:

- The PRIMARY KEY constraint uniquely identifies each record in a table.
- Primary keys must contain UNIQUE values, and cannot contain NULL values.
- A table can have only ONE primary key; and in the table, this primary key can consist of single or multiple columns (fields).
- Each database needs a unique identifier for every row of a table, and the primary key plays a vital role in identifying rows in the table uniquely.
- The primary key column can't store **duplicate values**.
- It is also called a **minimal super key**; therefore, we cannot specify more than one primary key in any relationship.

Database Design and ER Model: Constraints

PRIMARY KEY:

Syntax: In MySql

```
Create table <Table_name> (
        Column1 datatype(size),
        Column2 datatype(size),
        Column3 datatype(size),
      PRIMARY KEY (Column_name)
);
```

Example:

```
CREATE TABLE Student(
    Rollno int ,
     LastName varchar(255) NOT NULL,
     FirstName varchar(255),
     PRIMARY KEY (Rollno)
   );
```

Database Design and ER Model

- **Types of key:**
- **Primary key:**
- **For example:2**
- **Emp (Emp_Id, EmpName, salary)**
- **or**
- **Emp**

Emp_Id	EmpName	Salary
1001	Abhay	50,000
1002	Riyansh	30,000
1003	Vibha	40,000

Emp_Id is a primary key

Database Design and ER Model

Foreign key:

- A foreign key is generally a Primary key from one table that appears as attribute in the another table where the first table has a relationship with second.

Or

- F.K. is a column or combination of column in a table whose values should match a P.K. value in a other table.

- **Example:**
- Create two tables as **student** and **Dept**.
- Student(**RollNo**, Name, Class)
- Dept(DeptID, Dname, **RollNo**)
-
- **Student** will be a **parent** and **Dept** will be a **child**

Database Design and ER Model

Foreign key:

Example:

student

RollNo	Name	Class
1	Atherva	TE A
2	Jay	TE A
3	Bhupal	TE B

Dept

DeptID	Name	Class	RollNo
1	Atherva	TE A	1
2	Jay	TE A	2
3	Bhupal	TE B	3

Foreign key:

Example: for Foreign key in mysql

```
CREATE TABLE Dept(
        DeptID int NOT NULL,
        Dname Varchar,
        RollNo int,
        PRIMARY KEY (DeptID ),
        FOREIGN KEY (RollNo) REFERENCES Student(RollNo)
    );
```

Database Design and ER Model

- **Types of key:**

Super Key:

- is defined as a *set of attributes* within a table that can uniquely identify each record within a table.
- Super Key is a *superset of Candidate key*.

RollNo	Name	Phone	age
1	Aditya	7788990066	16
2	Ganesh	8899006633	17
3	Tanmay	9960334556	18
4	Aditya	9860667788	20

- In the table defined above super key would include:
- **RollNo**, (**RollNo**, **Name**), **phone** etc.

- **Super key:**

- **RollNo** is unique for every row of data, hence it can be used to identity each row uniquely.

- Next comes, (**RollNo**, **Name**), now **name** of *two students can be same*, but their **RollNo** can't be same hence this combination can also be a key.

- Similarly, phone **number** for every student will be **unique**, hence again, phone can also be a key.

- **Candidate key:**

- Relation schema may have more than one key in this each of key is called a candidate key.

- is a *set of attributes (or attribute)* which uniquely identify the tuples/row in relation or table.

- There is more than one candidate key can take place.

- A candidate key can never be NULL or empty.

- And its value should be unique.

Database Design and ER Model

- **Candidate key:**

- There can be more than one candidate keys for a table.

- A candidate key can be a combination of more than one columns(attributes).

- It is an attribute or a set of attributes that can act as a Primary Key for a table to uniquely identify each record in that table.

- **Example:**

- **RollNo & phone can be a candidate key**

Database Design and ER Model

- **Types of key:**

- **A composite key:**

- **Example:**

```
CREATE TABLE  Student (
   RollNo int NOT NULL,
    Name varchar(45),
    Class varchar(45),
    PRIMARY KEY(Name,  Class)
);
```

we have created a composite primary with the column names **Name** and Class.

Database Design and ER Model

- **Types of key:**

- **A composite key:**

- A composite key in MySQL is a *combination of two or more than two columns in a table* that allows us to *identify each row of the table uniquely*.

- It is a type of **candidate key** which is formed by more than one column.

- MySQL guaranteed the *uniqueness of the column only when they are combined*.

- If they have taken *individually*, the *uniqueness cannot maintain*.

- A composite key can be added in two ways:
- Using CREATE Statement
- Using ALTER Statement

Database Design and ER Model

Give difference between primary key and foreign key.

Key	Primary Key	Foreign Key
Basic	It is used to uniquely identify data in the table.	It is used to maintain relationship between tables.
Null	It can't be NULL.	It can accept the NULL values.
Duplicate	Two or more rows can't have same primary key.	It can carry duplicate value for a foreign key attribute.
Index	Primary has clustered index.	By default, It is not clustered index.
Tables	Primary key constraint can be defined on temporary table.	It can't be defined on temporary tables.

Database Design and ER Model

➢ Extended E-R Features: or Enhanced entity relationship

- Is a high level or conceptual data model.
- Extension to original ER model.
- The extended E-R features:
  - ✓ specialization,
  - ✓ generalization,
  - ✓ Aggregation,
  - ✓ higher- and lower-level entity sets,
  - ✓ attribute inheritance.

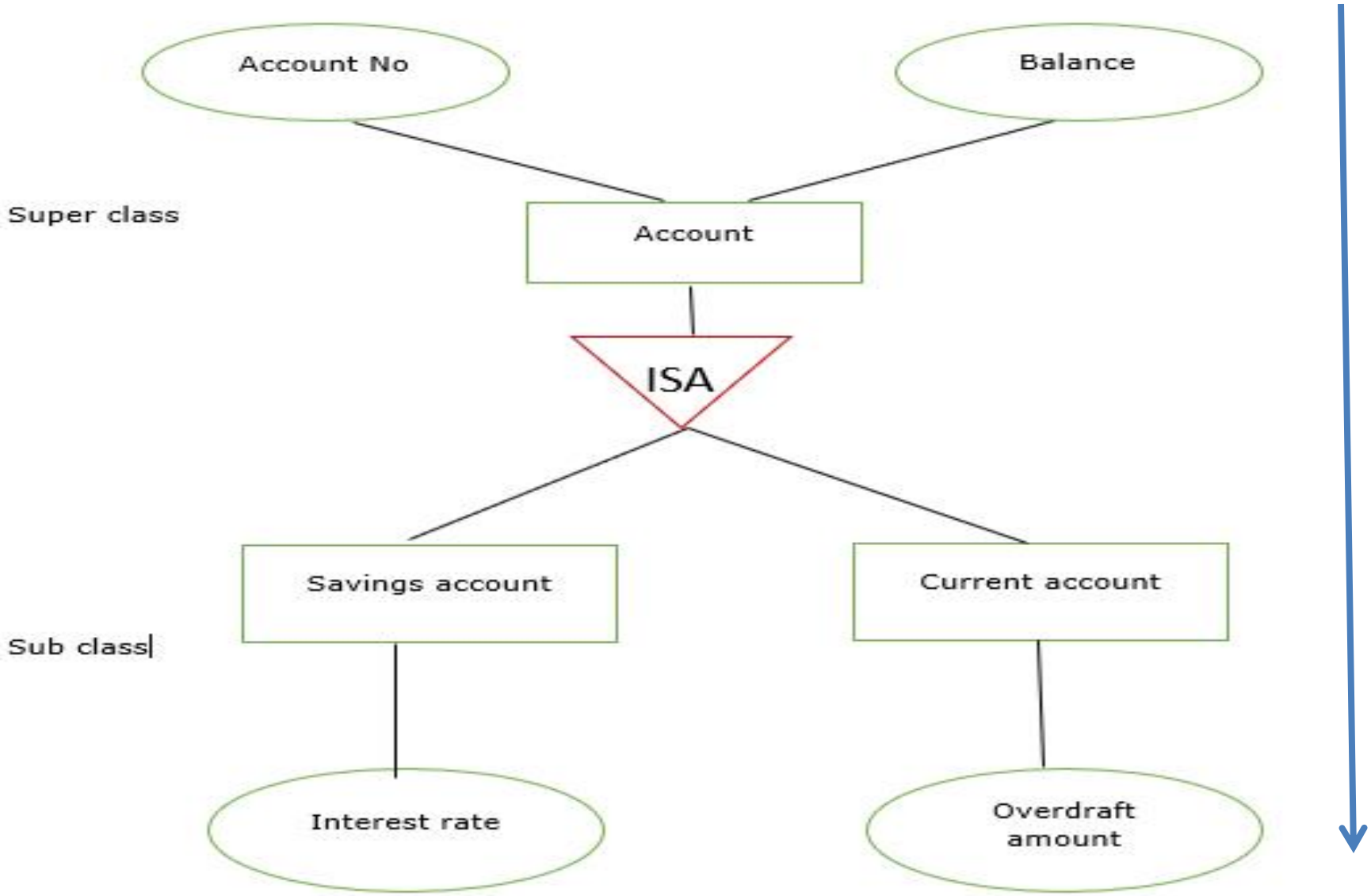
➢ Extended E-R Features: or Enhanced entity relationship

➢ Specialization:

- The process of ***sub groupings within an entity s***et is called specialization.
- It is a ***top-down*** process.
- one higher level entity can be ***broken down*** into two lower level entity.
- Entity are ***expanded***.

Database Design and ER Model

➢ Specialization: Example



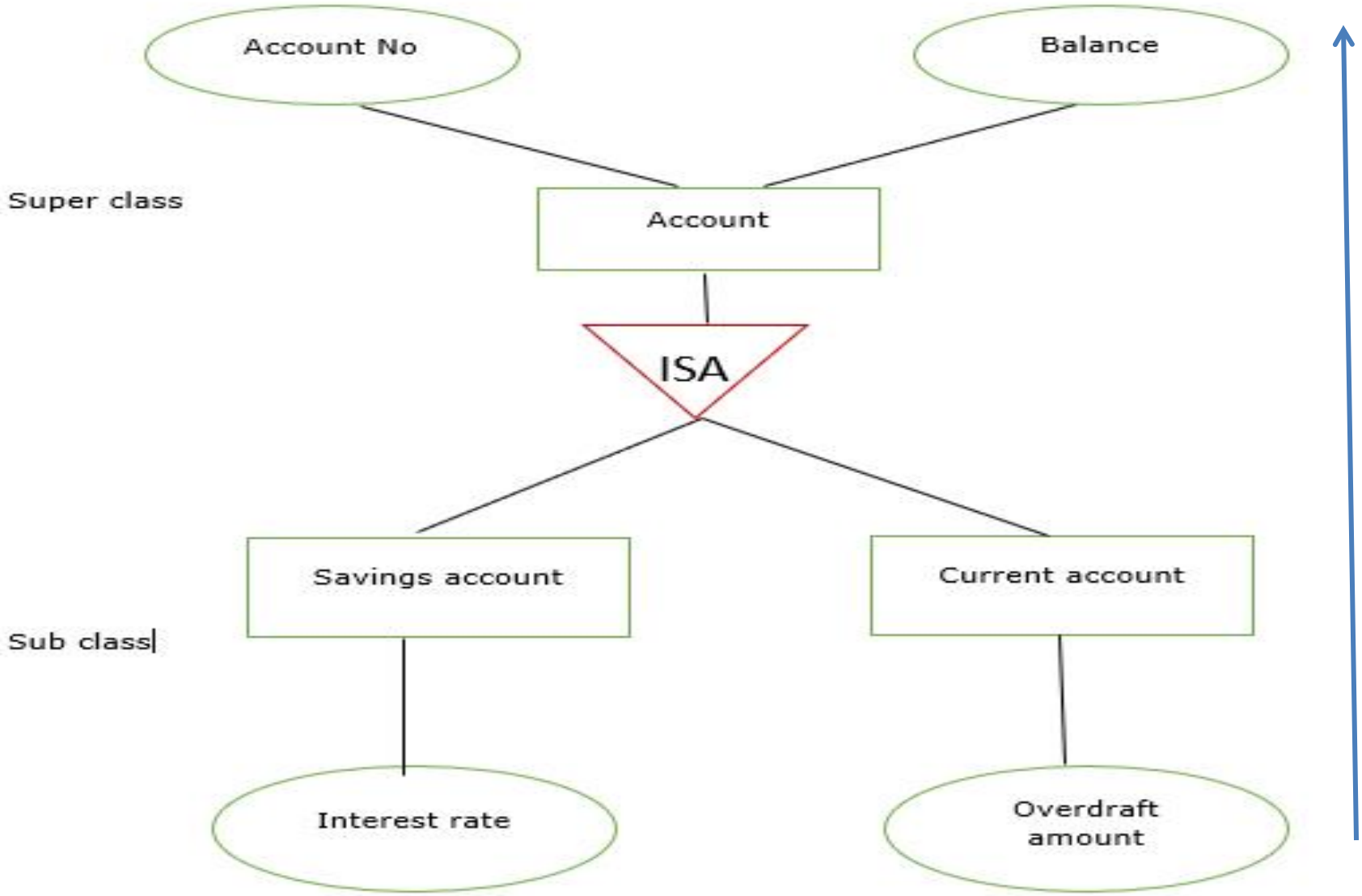
Extended E-R Features: or Enhanced entity relationship

Generalization:

- It is the ***reverse process of specialization***.
- It is a ***bottom-up*** approach.
- It converts ***subclasses*** to ***superclasses***.
- This process ***combines a number of entity sets*** that share the same features into ***higher-level entity sets***.
- It is a result of taking the ***union of two or more entity sets*** to ***produce a higher level entity set.***

Database Design and ER Model

➢ Generalization : Example



Database Design and ER Model

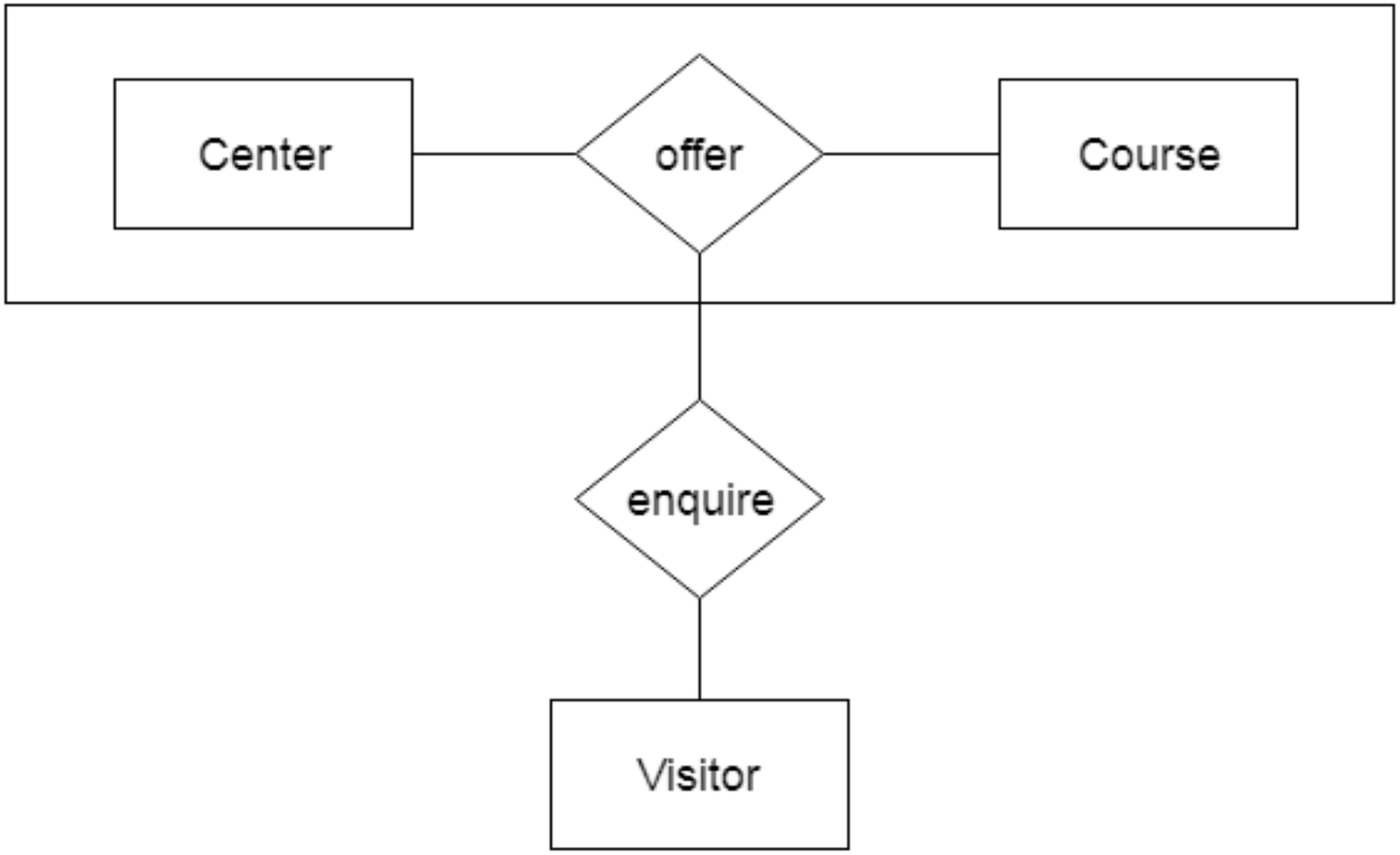
➢ Extended E-R Features: or Enhanced entity relationship

➢ Aggregation :

- The relation between two entities is treated as a ***single entity***.
- relationship with its corresponding entities is aggregated into a higher level entity.
- **For example**:
- Center entity offers the Course entity act as a single entity in the relationship which is in a relationship with another entity visitor.
- In the real world, if a visitor visits a coaching center then he will never enquiry about the Course only or just about the Center instead he will ask the enquiry about both.

Database Design and ER Model

- **Extended E-R Features: or Enhanced entity relationship**
  - **Aggregation :**



Integrity Constraints

1. Domain integrity constraints
    a. Not null constraints
    b. Check constraints

2. Entity integrity constraints
    a. Primary key
    b. Unique constraints

3. Referential integrity constraints
    a. References
    b. Foreign Key

4. On delete Cascade.

- Relational algebra
- Relational algebra operators:
- selection,
- projection,
- cross product,
- various types of joins,
- division,

- example queries,
- tuple relational calculus,
- domain relational calculus, and

- converting the database specification in E/R notation to the relational schema.

Relational algebra operators

- **Relational Algebra** is a **procedural query language** that works on relations (tables) and defines basic operations to manipulate and retrieve data.

- Relational algebra operators are the fundamental operations used in relational databases to manipulate data and retrieve information.

- These operators take relations (tables) as input and produce new relations as output.

- **Key operators include:**

✓ selection,
✓ projection,
✓ union,
✓ intersection,
✓ difference, Cartesian product, and join.

Relational algebra operators

- **Key Concepts in Relational Algebra:**

- Before explaining relational algebra operations, let's define some fundamental concepts:

- **1. Relations**: In relational algebra, a relation is a table that consists of rows and columns, representing data in a structured format.

- Each relation has a unique name and is made up of tuples.

- **2. Tuples**: A tuple is a single row in a relation, which contains a set of values for each attribute.

- It represents a single data entry or record in a relational table.

Relational algebra operators

- **Key Concepts in Relational Algebra:**

- Before explaining relational algebra operations, let's define some fundamental concepts:

- **3. Attributes**: Attributes are the columns in a relation, each representing a ***specific characteristic or property of the data.***

- For example, in a "Students" relation, attributes could be "Name", "Age", and "Grade".

- **4. Domains**: A domain is the set of possible values that an attribute can have.

- It defines the type of data that can be stored in each column of a relation, such as integers, strings, or dates.

Relational algebra operators

- **Selection operator (σ):**
- Selection operator is used to selecting tuples from a relation based on some condition.
- **Syntax:**
- $$\sigma_{(\text{Condition})}(\text{Relation Name})$$
- Extract students whose age is greater than 18 from STUDENT relation given in Table 3
- $$\sigma_{(\text{AGE}>18)}(\text{STUDENT})$$

Relational algebra operators

SELECT operation does not show any result, the projection operator must be called before the selection operator to generate or project the result.

❑ Projection:

- **Projection Operator (∏):**
- Projection operator is used to ***project particular columns from a relation***.
- **Syntax:**

$$\Pi_{(\text{Column 1,Column 2 ....Column n})}(\text{Relation Name})$$

- Extract **ROLL_NO** and **NAME** from **STUDENT** relation given in Table 3

$$\Pi_{(\text{ROLL_NO,NAME})}(\text{STUDENT})$$

Relational algebra operators

- If the resultant relation after projection has duplicate rows, it will be removed.

- **Example**:

$$\Pi_{(ADDRESS)}(STUDENT)$$

will remove one duplicate row with the value DELHI and return three rows.

Relational algebra operators

STUDENT_SPORTS

ROLL_NO	SPORTS
1	Badminton
2	Cricket
3	Badminton
4	Badminton

Relational algebra operators

2. EMPLOYEE

Roll. No	Name	ADDRESS	Phone no	AGE
1	RAM	DELHI	9455123451	18
2	NARESH	HISAR	9782918192	22
3	SWETA	RANCHI	9852617621	21
4	SURESH	DELHI	9156768971	18

3. STUDENT

ROLL_NO	NAME	ADDRESS	Phone no	AGE
1	RAM	DELHI	9455123451	18
2	RAMESH	GURGAON	9652431543	18
3	SUJIT	ROHTAK	9156253131	21
4	SURESH	DELHI	9156768971	18

Relational algebra operators

Display

$$\Pi\,(\sigma_{(AGE>18)}\,(STUDENT))$$

✓ Selects tuples from student where name is Ram' and address' is Delhi.

$$\sigma_{\text{name = “Ram" and address = “Delhi"}}(\text{student})$$

Relational algebra operators

$$\pi \text{ Name } (\sigma \text{ Marks} > 40 \text{ (Student)})$$

Relational algebra operators

- **Union (U):**

- Union on two relations R1 and R2 can only be computed if R1 and R2 are **union compatible.**

- These two relations should have the **same number of attributes** and corresponding attributes in two relations have the same domain.

- Union operator when applied on two relations R1 and R2 will give a relation with tuples that are either in R1 or in R2.

- The tuples which are in both R1 and R2 will appear only once in the result relation.

Relation1 U Relation2

- Find the person who is either student or employees, we can use Union operators like:

STUDENT U EMPLOYEE

Relational algebra operators

ROLL_NO	NAME	ADDRESS	PHONE	AGE
1	RAM	DELHI	9455123451	18
2	RAMESH	GURGAON	9652431543	18
3	SUJIT	ROHTAK	9156253131	20
4	SURESH	DELHI	9156768971	18
5	NARESH	HISAR	9782918192	22
6	SWETA	RANCHI	9852617621	21

Relational algebra operators

- **Minus (-) or Set Difference:**

- Minus on two relations R1 and R2 can only be computed if R1 and R2 are **union compatible.**

- Minus operator when applied on two relations as R1-R2 will give a relation with tuples that are in R1 but not in R2.

- **Syntax**:
- Relation1 - Relation2

Relational algebra operators

ROLL_NO	NAME	ADDRESS	PHONE	AGE
2	RAMESH	GURGAON	9652431543	18
3	SUJIT	ROHTAK	9156253131	20

Relational algebra operators

Rename(ρ):

Rename operator is used to rename a relation or its attributes for clarity or further operations.

Syntax:

ρ (new_relation_name, old_relation)

- To rename STUDENT relation to STUDENT_info, we can use rename operator like:

ρ(STUDENT, STUDENT_info)

Relational algebra operators

- **Cross Product(X):**
- Cross product is used to join two relations. For every row of Relation1, each row of Relation2 is concatenated.
- If Relation1 has m tuples and and Relation2 has n tuples, cross product of Relation1 and Relation2 will have m x n tuples.
- **Syntax:**
- **Relation1 X Relation2**
- To apply Cross Product on STUDENT relation given in Table 1 and STUDENT_SPORTS relation given in Table 2,
- **STUDENT X STUDENT_SPORTS**

Relational algebra operators

- **Cross Product(X):**
- Cross product is used to join two relations. For every row of Relation1, each row of Relation2 is concatenated.
- If Relation1 has m tuples and and Relation2 has n tuples, cross product of Relation1 and Relation2 will have m x n tuples.
- **Syntax:**
- **Relation1 X Relation2**
- To apply Cross Product on STUDENT relation given in Table 1 and STUDENT_SPORTS relation given in Table 2,
- **STUDENT X STUDENT_SPORTS**

Relational algebra operators

Set Intersection(∩)

Relational algebra operators